Lab #1 (basic bioelectricity)

How to work as a group

This lab involves three parts – coding/simulating, thinking about the questions, and writing a report. You only need to turn in one report per group.

The goal of having only one written report per group is to save you time in writing and save me time in reading. However, my expectation is that everyone in the group runs the simulations and discusses the questions together. The goal is *not* to have only one person learn the material 😊.

Overview:

In this lab, we'll

- get our first exposure to BITSEY (a friendly bioelectric simulator)
- run simulations of single cells to see what voltages they settle to

BITSEY is a smaller, simpler version of BETSE[1], meant specifically for classroom work. It's open-source Python code that you can find online, but it's easier to just grab it on the Halligan system from the link on the class web page. If you look at the BITSEY code, you will see five files:

- *main_sim_SS.py*: the main entry point. All of the functions that you are responsible for writing or modifying will go in main_sim_SS.py.
- *sim.py*: a library file containing the main bioelectric simulation routines.
- *edebug.py*: a library file with various debug-printing routines, to aid in figuring out why a simulation isn't giving you the results you want
- *eplot.py*: a library file with several nice routines that help make pretty plots of, e.g., cell voltage over time.
- *sim_toolbox.py*: a library file with some basic physics models (ion channels and pumps)

Instructions:

Please copy all five files to your own work directory, and then open main_sim_SS.py to take a look. If you're not familiar with running Python scripts, the course web page has a section on **Thonny and Python**, with short videos telling you how to get going with Thonny. Specifically, it tells you what to do about error messages about packages like *numpy* or *matplotlib* not being installed, and how to run a Python script with command-line parameters.

Let's run a simulation.

```
python main_sim_SS.py SS1 5
```

This says to run the Python script *main_sim1_SS.py*. This will call the function *setup_SS1*() to set up a simulation that instantiates four cells, and then simulates for 5 seconds of virtual time. For the moment, all four cells are identical. After a short simulation, it calls *post_SS1*() for any plotting that you like. It is originally set up to plot a graph of V_{mem} in each cell (and, since the four cells are identical, the four plots will all lie on top of each other). Feel free to experiment with changing the simulation time (via the command line) or the plots (by editing *main_sim_SS.post_SS1*()). The header of the file *eplot.py* lists some of the other plotting functions that may be useful.

Note the various debug data (such as the per-cell V_{mem}, ion concentrations and various other information) that also gets printed, both during the simulation and at the end. When debug information is printed, then typically each row (if there are multiple rows) is for one ion; each column is the data for one cell. See the documentation for the function *dump*() for more detail (at the top of the file *edump.py*).

Now that you know how to run a short simulation, it's time to do it for real. This time, we will run three simulations, each one for 100K seconds of virtual time (with **python main_sim_SS.py SS1 100000**). Here are the three simulations:

1. Exactly as you did at first, but now running for the full 100K seconds of virtual time. Since all four cells are identical, hopefully they behave identically – so their graphs will likely overlap.
2. Altered initial concentrations. This time, run **SS2** rather than SS1. Leave cell[0] the same (it will be your reference). Double the initial $[Na]_{ICF}$, $[K]_{ICF}$ or $[Cl]_{ICF}$ in cells [1], [2] and [3] respectively (i.e., each of those cells will have one ion concentration doubled). Remember to preserve charge neutrality by altering another of $[Na]_{ICF}$, $[K]_{ICF}$ or $[Cl]_{ICF}$ accordingly (but please do not touch [P]). Simulate for 100K seconds again.
3. Altered density of ion channels. This time, run **SS3**. As before, leave cell[0] the same (it will again be your reference). Double D_{Na}, D_K and D_{Cl} in cells [1], [2] and [3] respectively.

What to turn in:

Your version of main_sim_SS.py.

A lab report that contains several things. First, your graphs of V_{mem} for all three sims. For simulation #2, also turn in graphs of [Na], [K] and [Cl] across all cells. There's a call to *eplt.plot_ion*() already in the *post_lab1*() function; you should add K and Cl as well.

Finally, turn in the answers to the questions below in your report.

You need only turn in one report for each group.

Questions:

1. Turn in your plots (25 points).
2. At the end of each simulation, the function *dump*() dumps out various information about the system's final state. Note what it says about the flow rates (in moles/(m^3·sec)) for Na, K and Cl ion channels and pumps (and note that *dump*() skips printout of any zero-valued flow rates). Based on this information at the end of the sim, is the circuit essentially at steady state at the end of the simulation? Explain why, using the numbers for all three ions to show your point. (10 points)
3. Simulation #2 should show that your final steady-state results are insensitive to the cell interior's initial [Na], [K] and [Cl]. That's kind of amazing – and very useful, since it means that cells recover from random disturbances. We'll try to explore why this is. You may want to look at the plots of $[Na]_{ICF}$, $[K]_{ICF}$ and $[Cl]_{ICF}$ for cells #1-3 and compare them to cell #0.
    a. Our upcoming unit on QSS bioelectricity talks about negative feedback. In (e.g.,) cell #1, you increased $[Na]_{ICF}$. Ignore the fact that you also changed another ion to preserve charge neutrality, and focus only on the change in $[Na]_{ICF}$. How would

you expect increased $[Na]_{ICF}$ to (at least early in the sim) affect Na diffusion? By Q=CV, increasing $[Na]_{ICF}$ slightly changed V_{mem} for a short time; how would this have affected the Na drift flux? Can you argue that the combined effect of the new diffusion and drift fluxes tend to have a restorative effect (fighting back against the initial disturbance), or does it reinforce the original change? (10 points).

b. Would the cell-0 final SS solution also be a valid SS solution for cells #1-3? I.e., if cells #1-3 *somehow* eventually reach the same final cell-internal ion concentrations as does cell #0, can you argue that cells #1-3 would then be in steady state? (10 points).

c. (harder). After (a) and (b), you have hopefully argued that it is quite reasonable for cells #1-3 to return to the same point as cell #0. But can you argue that there is no other reasonable result? Use the hint below to make that argument. (5 points).

If you've made it through all three of these steps, then you've shown that (a) cells #1-3 have a tendency to move back to what cell #0 does, (b) when cells #1-3 do reach what cell #0 does, it will be SS for them too, and (c) all of this is not only believable, but mathematically correct.

4. Given the final values for $[Cl]_{ICF}$ and $[Cl]_{ECF}$ from simulation #2, compute V^{Nernst} for Cl. Does it roughly agree with your final V_{mem}? Explain why. (Note that the simulation results only show ICF concentrations and do not show ECF concentrations. To find ECF concentrations, look in the file *sim.py* for the function *init_big_arrays*(). You can see it initializing *c_out* for each ion, which is the default ECF concentration for that ion; BITSEY holds the ECF concentrations effectively constant through the sim). (10 points).

5. Simulation #3 should show that D_{Cl} does not affect your final results at all; that increasing D_{Na} makes the final V_{mem} more positive, and that increasing D_K makes the final V_{mem} more negative. Let's try to explain this. Remember that in BITSEY, *D* acts as a stand-in for the ion-channel density.

   a. Since Cl does not have a pump, it's especially easy to understand. Consider only cell #0, and assume it has reached steady state. Suddenly you double the conductivity of its Cl channels, effectively morphing cell #0 into cell #3. What happens to the Cl drift current? Cl diffusion current? If they were balanced before, are they still balanced? Can you use this to explain the results you saw in cell #3 vs. cell #0? (10 points).

   b. Now look at Na channels (cell #0 vs. #1) and K channels (cell #0 vs. #2). In cell #0, is the net (drift + diffusion) flow through the appropriate ion channel inwards, outwards or zero (see lecture slides 58-59)? Based on that, can you intuitively justify how doubling the relevant channel affected V_{mem} in cells #1 and #2? (10 points).

   c. In our equivalent-circuit model, the density of ion channels corresponds to the three parameters G_{Na}, G_K and G_{Cl}. Slides 14-17 of our Circuits Backgrounder show how to use our equivalent-circuit model of the cell to predict V_{mem}. Can you argue, based on this model, that increasing *D* should give the results that you saw? (In our next unit, we will see that this circuit model is more properly applicable to QSS than to SS, but for now we won't worry about the difference). (10 points).

A hint for question 3c. Let's say you built a set of equations and unknowns to compute the final cell voltage. You might have four variables (i.e., unknowns): the *final* $[Na]_{ICF}$, $[K]_{ICF}$, $[Cl]_{ICF}$ and V_{mem}. You might have one equation that says Q=CV (i.e., once you know [Na], [K] and [Cl] you automatically know V_{mem}). You might have another equation that says the total Na current is 0, and two similar equations for K and Cl. You can assume that a system of *N* equations in *N* unknowns always has one solution (even though this is not strictly true). What does that say about the initial $[Na]_{ICF}$, $[K]_{ICF}$ and $[Cl]_{ICF}$?

Summary of what we learned from this lab

Hopefully a few lessons come from this lab:

- The cell develops voltage in a way that's largely determined by its ion-channel conductances. This gives our bodies a nice way to change cell voltages.
- Even though ion concentrations can vary due to many disturbances, the cell robustly reaches whatever voltage the ion channels tell it. Transient changes to internal ion concentrations are just that – transient. The cell knows how to restore whatever ion concentration (and voltage) that the ion-channel conductances tell it.
- Simulation is easy and fun 😊

What's behind the curtain

You will notice that the very end of the file contains a call to the main function *command_line*(). This function first checks the command line to find out which simulation to set up and how long to simulate for. It then calls *sim.sim*() to actually run the simulation, *edb.dump*() to print the end-of-sim values, and finally calls the appropriate *post_* function to do any desired plotting.

Digging deeper

If you feel like digging a bit deeper, there's a debugging function *edb.analyze_equiv_network*() that looks at the current ion concentrations and builds an equivalent model just like we did in class. You can call it from *post_lab1*(). Does it give you the same V^{Nernst} for Na, K and Cl that you would calculate by hand? It should (at least, within a reasonable tolerance)! Does it predict the ion flow rates correctly (i.e., the same as reported by *dump*())? Remember that our linear model is just an approximation.

[1] *Bioelectric gene and reaction networks: computational modeling of genetic, biochemical and bioelectrical dynamics in pattern regulation*, Alexis Pietak 2017